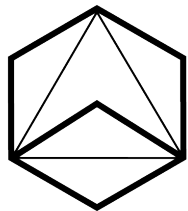




Golf Cart bring-up progress



THE
AUTWARE
FOUNDATION

National Taiwan University

Autware LSV meeting, August 2026

國立臺灣大學



嵌入式系統暨無線網路實驗室



Vehicle platform

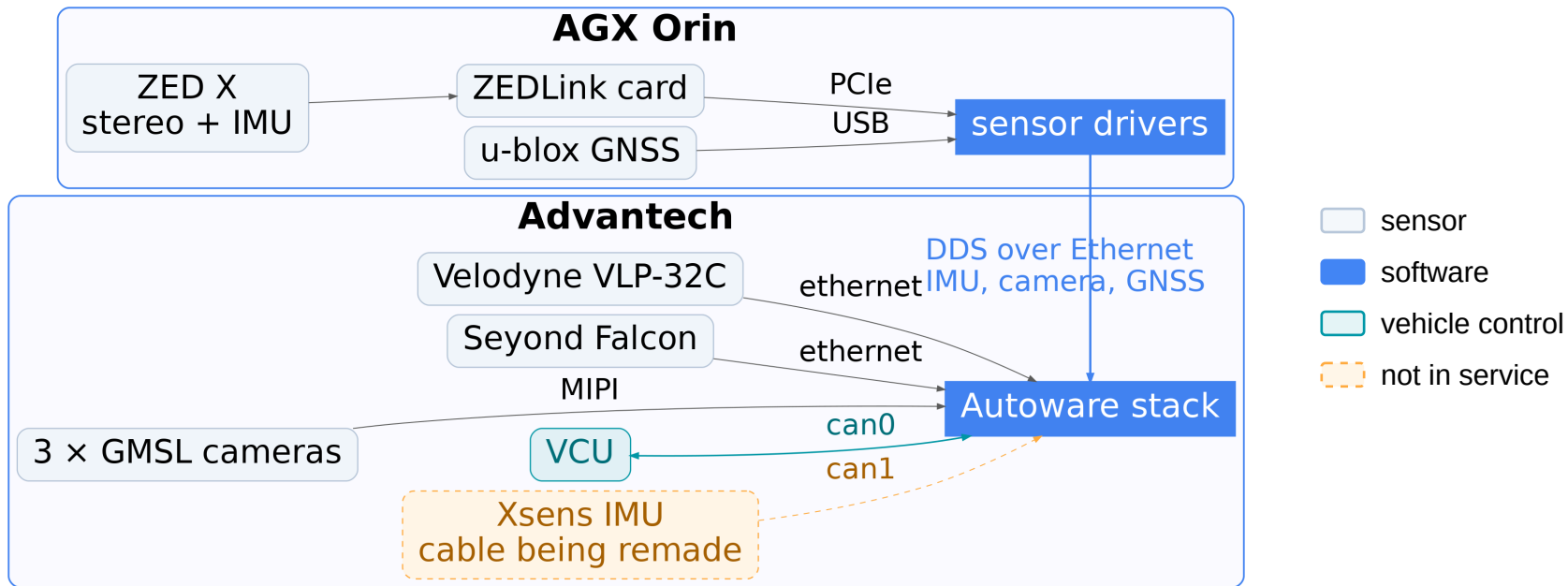


A low-speed campus vehicle on a **Turing Drive VCU**, with an Autoware stack across **two machines**.

Bring-up sequence:

1. Sensors ready
2. Two-host system ready
3. Vehicle interface in progress
4. Map preparation ready
5. Data collection done, data flawed
6. Autonomous run not started

System architecture



Sensor integration

All sensors publish, across both hosts.

Sensor	Host	Feeds
Velodyne VLP-32C	Advantech	NDT scan matching, perception
Seyond Falcon	Advantech	perception
3 GMSL cameras	Advantech	perception, recording
ZED X stereo + IMU	Orin	the IMU the stack runs on today
u-blox GNSS	Orin	pose initialisation
Turing Drive VCU	Advantech	velocity, gear, subsystem states

Multi-host orchestration

ROS 2 provides no equivalent of ROS 1's machine tag, so orchestration is ours to supply. It is in place and operational.

Single instance, no orphaned processes

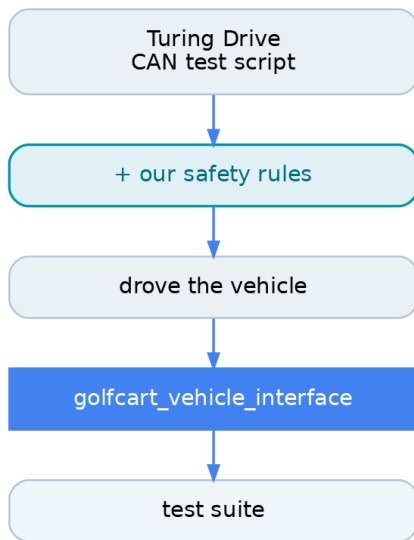
- **play_launch** reclaims orphaned processes when a launch terminates, and brings the stack up in **20 s** against **60 s** for `ros2 launch`
- **systemd user units** around it: singleton by construction, `KillMode=control-group` takes the whole tree down

Uniform environment on either host

- one CycloneDDS profile per role, plus a `config/host` marker file
- `scripts/env.sh` reads it, and every shell and every unit sources it
- so any shell is correctly configured on open, and no operator needs to track which host they are working on

`just launch-all` starts both hosts and returns; `just stop-all` shuts them down.

Vehicle interface



Developed from the vendor's own test code rather than from a specification. Every stage below exists in the repository.

The safety constraints are our addition:

- target speed is clamped non-negative; reverse is gear R
- gear P forces speed and steering angle to zero, re-applied each cycle
- ESTOP release is never bound to a key the terminal cannot emit

A test suite sits above it: keyboard control, a command service, and trajectory replay.

Data acquisition



Three runs at NTU. **Each host records independently**, writing only the topics for the devices it owns; the bags are merged afterwards.

Only **first-hand driver output** is recorded. Derived topics are excluded, so replay recomputes them under current parameters rather than those fixed at record time.

Replay is a single command, just `ntu-test run: bag paused for /clock`, then `stack`, `RViz` and initial pose, in the required order.

Integration findings

Three findings relevant to comparable deployments.

- **The vendor's default oToCam setup enables the cameras and disables every USB port.**
A workaround now provides both. The .ko files remain ABI-bound to kernel 5.15.148-tegra, so a JetPack update silently reverts it.
- **The Xsens IMU cable has failed and is being remade.** The stack currently uses the **ZED X built-in IMU**, which resides on the other host and crosses the network at 100 Hz.
- **The u-blox GNSS is hosted on the Orin**, the Advantech having no free USB port. A second localization input is therefore also remote, for an unrelated reason.

Compute and thermal limits

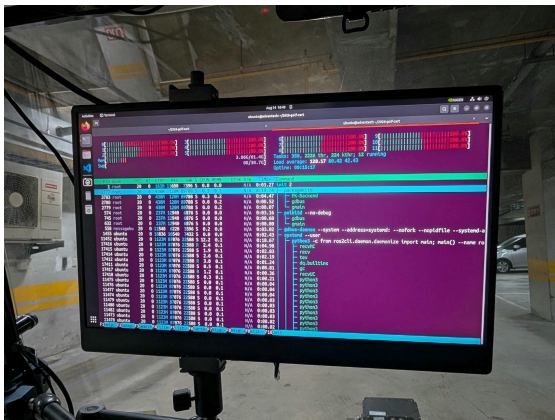
The three GMSL cameras emit **UYVY** at 1920x1280, 30 fps, while the consuming nodes require **RGB or JPEG**. A conversion is therefore unavoidable. **nvvidconv** was evaluated; the conversion still incurs CPU cost per camera.

This load contributes to the thermal saturation shown here, and is part of the rationale for splitting the stack across **two hosts**.

Mitigation in progress: `gmslcam`, which removes that stage.



Start-up contention



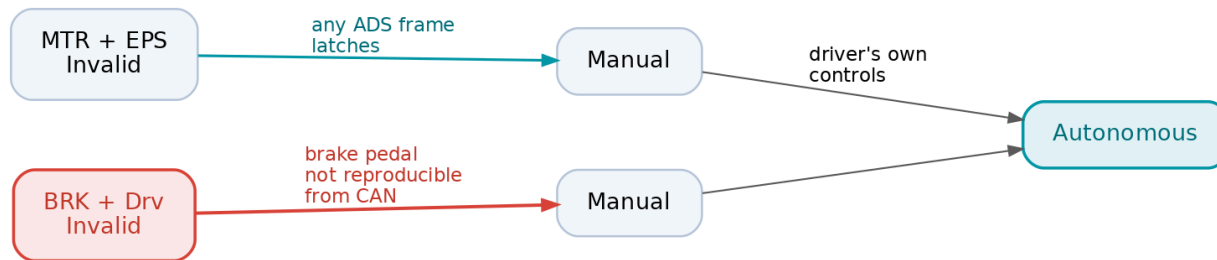
`play_launch` is fast *because* it spawns without rate limiting. On a host already at its limit this produces a thundering herd, and the machine becomes unresponsive, requiring a power cycle.

Rate-limiting the spawns is the obvious remedy, and measurement rejects it: roughly 10% fewer runnable tasks for more than double the start-up time, forfeiting the advantage the tool was adopted for.

The shipped mechanism is a **1 GiB MemAvailable floor**, engaging only as the host approaches exhaustion.

Constrain the resource that is exhausted, not the spawn rate.

Autonomous engagement and VCU state entry

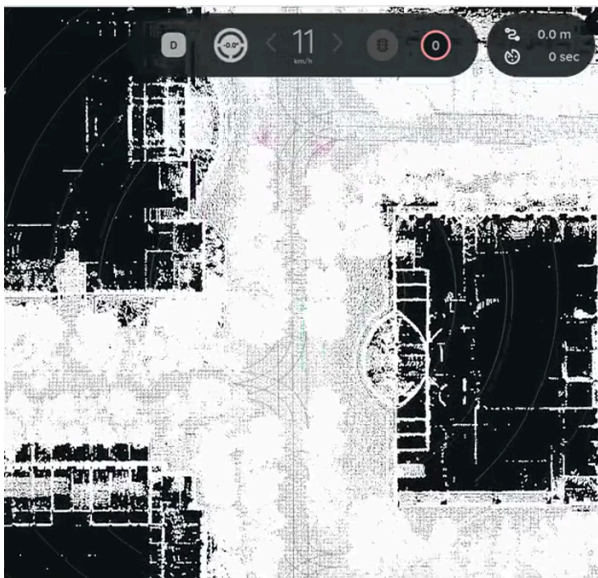


There is no `vehicle_cmd_gate` external selector. The interface issues no commands until all four subsystems report autonomous, and **no service call can override this**.

After a VCU restart, **BRK and Drv initialise as Invalid**, and only a brake-pedal press was observed to clear them. Not reproducible over CAN.

Unattended autonomous start-up is therefore blocked. This requires clarification from the vendor rather than further work on our side.

NDT localization



Scan aligned to the map at 11 km/h. Clip available at 2x.

The first successful NDT initialisation and tracking run on the NTU map. Initialisation from GNSS, and convergence to 0.14 m while stationary.

Tracking then **degrades within seconds of the vehicle moving**. Tuning was evaluated against **scan-to-map residual** rather than the NVTL score: the change that most improved accuracy in fact **reduced** NVTL.

The cause has been identified, and lies upstream of **NDT in the recording**: 45% of scans cover only part of a revolution, and scans arrive **376 ms stale**. Neither is a localization parameter.

Both are addressed in the next collection run, which records raw LiDAR packets.

Bring-up status

Step	State	Outstanding issue
Sensors	ready	partial LiDAR scans; Xsens cable under repair
Two-host system	ready	none
Vehicle interface	in progress	the VCU will not enter its autonomous state over CAN alone
Map preparation	ready	PCD and lanelet2 supplied by Turing Drive, downsampled at runtime
Data collection	done, data flawed	the recordings carry the sensor defects above
Autonomous run	not started	dependent on the vehicle interface issue

Next steps

- **Establish with Turing Drive what clears the Invalid brake state after a restart.** All remaining work on the autonomous run depends on this.
- **Record raw LiDAR packets** and verify the VLP-32C rotation configuration against the hardware, then repeat the localization work on sound data.
- **Migrate the sensor kit to gmslcam**, eliminating the per-camera CPU conversion.

TSN is a separate line of work, available for discussion if of interest.